Lab prático — Funções e a pilha (Unidade 4)

Lab **no computador**: agora que você tem **funções**, **passagem por referência** (`*` / `&`), **recursão** e entende a **pilha de execução**, vai construir e rodar programas **modulares**.

Como fazer

- **Um arquivo por exercício** (ex.: `ex1.c`). Compile e rode, sem warnings:

```
gcc -std=c11 -Wall -Wextra ex1.c -o ex1 && ./ex1
```

- **Use as assinaturas pedidas** (os nomes/parâmetros indicados) — assim dá para conferir.
- **Escopo:** funções (protótipo, parâmetros, retorno), passagem por valor/referência, recursão, pilha e compilação separada — além de tudo das Unidades 0–3 (tipos, E/S, operadores, `if` /laços).
- **Boa prática (vale nota):** funções de **cálculo não fazem** `printf` / `scanf` — quem lê e escreve é o `main` .

Nível 1 — Fácil

1. Biblioteca de funções puras

Implemente **duas funções que só calculam e retornam** (sem imprimir nada dentro delas), e um `main` que lê e chama:

- `int maior(int a, int b)` — devolve o maior dos dois.
- `long fatorial(int n)` — devolve `n!` (pode ser com laço).

Exemplo:

```
Dois inteiros: 7 3
Maior: 7
n para o fatorial: 5
Fatorial: 120
```

Pratica: protótipo, parâmetros, retorno, e **separar cálculo de E/S**.

2. Troca por referência

Implemente `void troca(int *a, int *b)` que **troca** os valores de duas variáveis. No `main` , leia dois inteiros, chame `troca(&x, &y)` e imprima os dois depois.

Exemplo:

```
Dois inteiros: 2 9
Depois: x = 9, y = 2
```

Pratica: passagem **por referência** (`*` no parâmetro, `&` na chamada). **Pergunta:** por que isso **não** funcionaria se `troca` recebesse `int a, int b` (por valor)?

Nível 2 — Médio

3. Dois resultados de uma vez (por referência)

Uma função só pode `return` um valor — mas, por **referência**, pode devolver vários. Implemente `void divmod(int a, int b, int *q, int *r)` que calcula o **quociente** (em `*q`) e o **resto** (em `*r`). No `main`, leia `a` e `b`, chame `divmod(a, b, &q, &r)` e imprima.

Exemplo:

```
Dividendo e divisor: 17 5
Quociente: 3, Resto: 2
```

Pratica: por que o `scanf` usa `&` — é exatamente isto: a função escreve no endereço que você deu.

4. Recursão × iteração

Implemente a soma `1 + 2 + ... + n` de **duas** formas:

- `long soma_rec(int n)` — **recursiva** (identifique o **caso-base** e o **passo recursivo**).
- `long soma_iter(int n)` — **iterativa** (com laço).

No `main`, leia `n` e imprima as duas (devem dar o mesmo).

Exemplo:

```
n: 100
rec = 5050, iter = 5050
```

Pratica: recursão. **Pergunta:** qual versão usa **mais memória** e por quê? (Pense na pilha: quantos quadros a recursiva abre ao mesmo tempo?)

Nível 3 — Difícil

5. Segurança: conserte o `gets`

O código abaixo lê um nome num `char buf[8]` usando `gets` — **perigoso** (aliás, o compilador moderno **nem aceita** `gets`!). **Reescreva a função `echo` de forma segura** usando `fgets`, e teste digitando um texto **bem maior que 7 caracteres**: o programa **não** pode quebrar.

```c
#include <stdio.h>

void echo(void) {
    char buf[8];
    gets(buf);                      // PERIGOSO: nao limita o tamanho
    printf("Voce digitou: %s\n", buf);
}

int main(void) {
    echo();
    return 0;
}
```

Pratica: buffer overflow e entrada segura. **Pergunta:** o que, na **pilha**, o `gets` poderia sobrescrever se você digitasse demais?

Nível 4 — Muito difícil (Interface H/S)

6. Compilação separada + ache o `call` / `ret`

Separe o programa em **três arquivos** e compile cada `.c` por si, ligando no fim:

- `mathlib.h` — os **protótipos** de `maior` e `fatorial` (do exercício 1).
- `mathlib.c` — as **definições** dessas funções.
- `main.c` — um `main` que `#include "mathlib.h"` e chama as duas.

Compile e ligue:

```
gcc -std=c11 -Wall -Wextra -c mathlib.c   # gera mathlib.o
gcc -std=c11 -Wall -Wextra -c main.c      # gera main.o
gcc mathlib.o main.o -o app               # o LIGADOR junta os dois
./app
```

Depois, **olhe o código de máquina** e ache as instruções que **entram** e **saem** de uma função:

```
objdump -d app      # procure, dentro de <main>, a instrucao 'call' (chamada) e a 'ret' (ret
```

Pratica: o papel do **ligador** (compilação separada) e o **gancho hardware/software**: `call` empilha o endereço de retorno; `ret` desempilha. **Pergunta:** o que acontece se você esquecer `mathlib.o` no comando de ligação? (Experimente e leia o erro.)

Para entregar / discutir

1. No exercício 2, explique por que a troca **só** funciona por referência.
2. No exercício 4, com `n = 5`, **desenhe os quadros** da `soma_rec` sendo empilhados e desempilhados (como no interativo da unidade).
3. No exercício 6, cole o erro que aparece quando você **esquece** `mathlib.o` na ligação e explique, em uma frase, o que o ligador estava reclamando.